

MetaCluster: A Universal Interpretable Classification Framework for Cybersecurity

Wenhan Ge^{ID}, Zeyuan Cui^{ID}, Junfeng Wang^{ID}, Binhui Tang^{ID}, and Xiaohui Li^{ID}, *Member, IEEE*

Abstract—Rising cyber threats have created an immediate demand for Deep Learning (DL) in cybersecurity. Nevertheless, the opaque nature of DL models poses challenges in deploying, collaborating, and assessing their effectiveness in less reliable cybersecurity environments. Despite eXplainable Artificial Intelligence (XAI) playing a role in enhancing cybersecurity analytics, the limited task scope, the propensity for data overfitting, and the stochastic explanations hinder its broader application. To fill the gap, this paper introduces a generic interpretable classification framework, named MetaCluster. MetaCluster generates semantic prototypes for features, patterns, and domains at varying granular levels by following three fundamental steps: embedding representations, acquiring prototypes, and aggregating semantics. These mechanisms guarantee that MetaCluster achieves critical information extraction and reliable classification at minimal cost. The experiments encompass cybersecurity classification tasks and assess the interpretability of the framework. These tasks encompass malware family classification, threat behavior analysis, and malicious traffic identification. In particular, when compared to other DL models, MetaCluster exhibits a significant reduction in parameter consumption by 79.52% to 91.78%, and boosts operational speed up to 71.37%, while its F1 scores remain stable or slightly increase. Additionally, MetaCluster possesses the ability to assess and visually represent the significance of image, text, and statistical features. This capability leads to a reduction of Mean Squared Error (MSE) between expected and actual predictions by 0.0101 to 0.1020.

Index Terms—eXplainable Artificial Intelligence (XAI), cybersecurity, interpretable classification, model lightweight, general framework.

I. INTRODUCTION

THE escalating cyber threat landscape has inflicted substantial losses on internet applications, ranging from data breaches and service outages to facility failures. According to

Fortinet's report [1], ransomware variants saw a nearly 100% increase in 2022 compared to 2021, resulting in losses of \$600 million in the first half of 2021. The primary impediment to efficient and effective cyber threat analysis lies in the disparity and asymmetry between cyber attackers and defenders. This inequity is evident in the discrepancies in time and cost, the uncertainty surrounding attack occurrences, and cognitive biases [2]. It's worth highlighting that Deep Learning (DL) approaches can rectify this imbalance. They can automate processes to reduce costs, enhance threat perception, and leverage multi-source Cyber Threat Intelligence (CTI) to enrich insights [3], [4]. However, there are advantages and disadvantages just as there are two sides to the sword. Adversaries can employ countermeasures, such as using adversarial samples to evade detection [5], [6], or injecting backdoor samples to compromise and steal data [7]. Therefore, **there is an urgent demand for cybersecurity methods that can be detected, monitored, and trusted** [8].

Researches in the field of explainable cybersecurity analysis typically involve three methodologies: reliance on general frameworks, the use of attention mechanisms, and the application of rule generation. **General frameworks** in cybersecurity [9], [10], [11], [12], [13], [14], [15] often consist of simplified network approximations like LIME [16] and SHAP [17]. These frameworks may also incorporate adversarial-based feature sampling [18], [19]. General frameworks offer versatility across multiple tasks but may yield unstable outcomes, reduced precision, and the need for additional parameters or training. **Attention mechanisms** [20], [21], [22], utilizing attention networks, focus on the influence of distinct features on subsequent tasks. They often require supplementary visualization tools to clarify the extent of feature focus [23], [24], [25], [26]. Unlike general frameworks, attention mechanisms are relatively straightforward, as their interpretability is inherent within the original model. However, complex models like GPT [27] and GLM [28] can lose interpretability, leading to epistemic illusions. **Rule generation** is an intuitive approach involving the translation of features into human-readable language rules to formulate security decisions. Decision Trees (DT) [29], [30], [31] and detection rules [32], [33], [34], [35] fall into this category. While rule generation is transparent, the alignment with human comprehension may not always match machine suitability [36]. Consequently, rule generation often leads to precision loss and issues related to boundary ambiguity. Additionally, prior

Manuscript received 27 October 2023; revised 26 January 2024; accepted 22 February 2024. Date of publication 4 March 2024; date of current version 2 May 2024. This work was supported in part by the National Natural Science Foundation of China under Grant U2133208 and Grant 62101368, in part by the Sichuan Youth Science and Technology Innovation Team under Grant 2022JDTD0014, and in part by the Sichuan Science and Technology Program under Grant 2022YFG0168. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Andrew Clark. (*Corresponding author: Junfeng Wang.*)

Wenhan Ge, Zeyuan Cui, and Junfeng Wang are with the College of Computer Science, Sichuan University, Chengdu 610065, China (e-mail: wangjf@scu.edu.cn).

Binhui Tang and Xiaohui Li are with the School of Cyber Science and Engineering, Sichuan University, Chengdu 610065, China.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIFS.2024.3372808>, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2024.3372808

knowledge of engineering significantly influences the method's performance.

Based on the aforementioned summary, it indicates that the interpretability of cybersecurity is still in its early stages of development. There are three main reasons for this:

- a) **Single-Task Limitation:** Some explanation models may not be universally applicable to multi-tasks, as they primarily focus on interpreting the specific data structures in input features. Even the most generalized versions of LIME and SHAP may pose challenges and mistakes when using the TensorFlow2.0 framework.
- b) **Data Overfitting:** Issues such as data imbalance and the black-box nature of neural networks can introduce noisy semantics along unknown pathways, disrupting the classification of normal semantics and leading to parameter redundancy, causing the model to become overfitted to the current dataset [37].
- c) **Stochastic Explanation:** Due to factors like stochastic gradient descent and stochastic initialization, existing explanation methods may not always yield reliable and stable results [38]. At the same time, a large number of explanations rest on human identification [25], [39] and lack quantitative assessment indicators.

Therefore, it is imperative to prioritize a generalizable framework ensuring the lightweight and highly accurate classification and quantitative interpretation for cybersecurity interpretive analysis.

To address this gap, this paper introduces MetaCluster, a general classification framework based on semantic prototype aggregation. MetaCluster is designed to enhance interpretability across various cybersecurity tasks and address the following Research Questions (RQ):

- RQ1.** How to establish a **lightweight, highly accurate, generalizable, and interpretable framework** for all kinds of cybersecurity classifications?
- RQ2.** How to **quantitatively assess the interpretation** of results for a more reliable model that meets expectations?

Firstly, MetaCluster transforms any input data type into 3-dimensional vectors through embedding operations to answer RQ1. Secondly, inspired by the Prototype Network (PN) [40] of meta-learning, MetaCluster assumes that there exists a set of semantic prototypes of each granularity feature to guide its contribution to the classification process. Therefore, MetaCluster repeats the three basic steps of "embedding features, finding prototypes, and aggregating similar features" to filter and merge the features of each network layer in the classification process into high-level features, forming a prototype decision process. Finally, to answer RQ2, MetaCluster provides the Mean Squared Error (MSE) as a measure of the gap between the model prediction and the standard expectation when using main and none semantics. The contributions of this paper are as follows:

- 1) This paper proposes a generalized interpretive framework for explaining most of the cybersecurity classification tasks, which can be used to derive the contribution of arbitrary input features to the classification results and produce highly accurate and efficient recognition results.
- 2) A set of assessment methods is designed to verify the quality of the results produced. By forming prototypes of the categories, we validate the ability of models to differentiate key information. And, by inputting blank noise, the models have been tested to resist noise interference.
- 3) We conduct experiments with multiple data sources from multiple scenarios. The results show that our model compresses a large portion of computational space and time while guaranteeing highly accurate recognition. It is believed that the method in this paper can be adopted for richer scenarios.

The paper is structured as follows: Section I provides an overview of the need for interpretative analysis in network security and outlines the study's contributions. Section II explores the relevant literature on prototype networks and similar models in the context of interpretability research. Section III elaborates on the methodology and its implementation in this study. Section IV covers the empirical experimentation phase, including classification and explication tasks across various network security analysis scenarios. Finally, Section V summarizes the entire manuscript comprehensively.

II. RELATED WORK

A. Explainable Artificial Intelligence

Before diving into a detailed presentation of relevant work, it's essential to clarify the concept of eXplainable Artificial Intelligence (XAI). XAI can be broadly categorized into two key aspects: global interpretability and local explainability [41].

Global interpretability entails understanding the algorithmic logic underlying a neural network model. This type of interpretability aligns with natural laws and human reasoning processes. Models like Decision Trees (DT) [42], naive Bayes [36], [43], Markov decision processes [44], and Monte Carlo decision-making [45] adhere to this approach.

Local explainability, on the other hand, focuses on explaining individual examples by assessing the importance and contribution of features and highlighting the relationship between features and the model's decision. Local explainability emphasizes the input-output relationship, often neglecting the internal logic of the network's decision-making process. This makes it well-suited for most black-box models. However, this focus on individual instances can introduce challenges such as result instability, dependence on sample quality and difficulty in establishing a unified basis for generalization [18], [19], [20]. Typical methods for local explainability include LIME [16], which modifies feature representations to observe corresponding changes in outcomes, and SHAP [17], which explores the contribution of groups of features to outcomes through feature games.

B. XAI for Cybersecurity

As discussed in Section I, current efforts in interpreting cybersecurity can be broadly categorized into three cases. These include: the **Case explanations** using a general explanatory framework, the **Attention mechanisms** with embedded

feature attention modules, and the **Rule generation** that produces rules comprehensible to humans. The first two methods of these are currently the mainstream approaches, as they are effective and do not necessitate complex expert knowledge integration.

1) *Case Explanation*: In the field of cybersecurity, the primary XAI frameworks are mainly LIME and SHAP. For instance, Alani [9] and Kinkead [10] respectively applied SHAP and LIME in malware analysis. Suryotrisongko [12] used both LIME and SHAP for botnet identification, while Psychoula [13] employed LIME and SHAP for fraud detection. Moreover, LIME and SHAP have found applications in other domains such as spam file retrieval [11] and network intrusion detection [15], [46], where their explainability features have been leveraged.

Further, more researches have been done to emphasize information variability based on LIME and SHAP. TRUST-XAI [37] statistically counts the most important variables by assuming Gaussian distributions and input-output information entropy. SeqMask [25] measures the impact of different feature masks on the classification results by counting the change in KL scatter. SATG [20] and Aditya [47] respectively use an adversarial-based approach to explore and explain the boundaries of understanding attack cases and textual semantics.

While the utilization of a nested XAI framework provides explanatory foundations for most tasks, its instability often necessitates additional expert analytical procedures [18], [19], [20]. Therefore, there exists a growing demand for an XAI framework that enables the transfer of explanatory assessment feedback to the original model.

2) *Attention Mechanism*: Self-interpretable attention mechanisms present another avenue in XAI. The integration of attention empowers neural networks to naturally revisit contextual relationships and determine feature importance.

The Transformer methodology [23], [24], [26] introduced attention to the analysis of malware and CTI, resulting in improved accuracy. SATG [20] explores the contributions of the self-adversarial central attention mechanism in recognizing sufficient and necessary behavioral patterns during attacks. ViT-NeT [22] integrates prototype match into visual transformers to facilitate the segmentation of salient features in images.

Attention mechanisms introduce a bias in feature selection, but an excessive accumulation of attention layers can complicate subsequent explanations. Large-scale language models like GPT [27] and GLM [28], which adhere to the Transformer architecture, are still in the early stages of determining their underlying reasoning processes.

3) *Rule Generation*: Rule generation is a method that can represent the model's decision-making process comprehensively, with the DT being a prominent example of the way. Mahbooba [29] utilized DT as an explanatory approach in intrusion detection systems. Similarly, Wang et al. [30] employed the DT method for mobile malware traffic detection. Verkerken et al. [48] also used Random Forests (RF) as a layered detector in unknown attack detection.

More advanced approaches such as Neural DT [49], Deep Neural DT [31], and Neural Decision Forest [50] use neural networks to determine the decision tree's partition

domains, guiding inputs with varying distributions into different branches for classification.

While rule generation allows neural networks to categorize feature combinations with similarities, this tree-like partitioning can affect the model's decision-making process, especially in cases of sequences with varying lengths and image categorization, particularly in multi-label classification. ProtoPNeT [51] introduces semantic prototypes to filter features and replace strongly segmented regions of input features, resulting in softer importance scores and improved model applicability and generalization. ProtoTree [52] and ViT-NeT [22] extend ProtoPNeT's single-level semantic prototype comparison into a binary comparison tree to distinguish between critical and minor features. This deepens the model's decision pathways but also increases resource consumption.

4) *Summary*: Although the XAI in cybersecurity analysis is in its early stages, the integration of deep learning and interpretability methodologies has become a pivotal aspect of XAI research. This paves the way for the exploration of multifaceted cybersecurity XAI frameworks as an inevitable trajectory in this field.

The core concept of MetaCluster lies in its bottom-up one-way decision graph with granularity-by-granularity generalization. In contrast to traditional DT structures, this approach permits the sharing of sibling nodes. It places a greater emphasis on preserving the matching and filtering of feature prototypes at each level, rather than relying on the common attention mechanism. Differing from expert matching rules, MetaCluster generates a step-by-step granularity of feature importance distribution for DL.

III. METHODOLOGY

As discussed in **Sect. I**, MetaCluster primarily addresses the questions related to RQ1: Universal Classification Framework and RQ2: Quantitative Interpretation Assessment. The RQ1 will be addressed in **Sect. III-A**, while the RQ2 will be answered in **Sect. III-B**.

A. MetaCluster Model

Ensuring interpretability of the process entails making every neural network layer interpretable, while the lightweight model requires minimizing noise semantics as feasible. By combining the characteristics of graph learning and PN, MetaCluster converts all embedding operations throughout the classification process into prototypical network operations. It utilizes the similarity of "layer prototypes" as guidance for "graph aggregation", creating aggregated features that are involved in subsequent embedding transformations. *As a result, each layer of the MetaCluster neural network comprises three core steps: feature embedding, meta-generation, and meta-aggregation.* The structure of the MetaCluster Model is in **Fig. 1**. The formal expressions for these steps are as follows:

$$x_t = g_t(x_{t-1}) = \begin{cases} f_t(l_t(x_{t-1}, p_t) \cdot x_{t-1}) & (1a) \\ l_t(x_{t-1}, p_t) \cdot f_t(x_{t-1}) & (1b) \end{cases}$$

where $x_t \in \mathbb{R}^{|x_t| \times v_t}$ represents the output of layer t as well as the input for layer $t+1$. $f_t(\cdot) : \mathbb{R}^{|x_{t-1}| \times v_{t-1}} \rightarrow \mathbb{R}^{|x_t| \times v_t}$ denotes

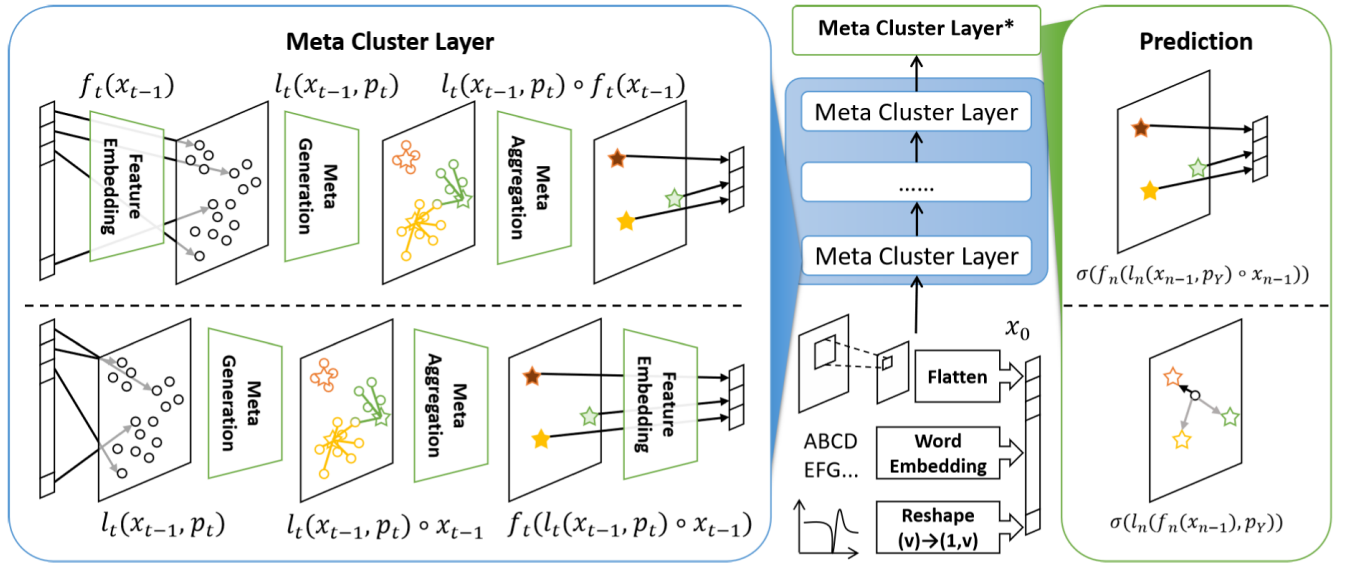


Fig. 1. MetaCluster model structure. MetaCluster accepts many types of input, such as images, text, and tables. It transforms the input into a formal input model in the form of $\mathbb{R}^{w \times v}$. Each layer of MetaCluster follows the three basic steps of “feature embedding, prototype finding, and semantic aggregation” to form a progressively constrained feature filtering mechanism to predict categories.

the embedding method, which can be any standard neural network layer. $p_t \in \mathbb{R}^{|x_t| \times v_{t-1}}$ corresponds to the semantic prototype for each layer. In the original PN, semantic prototypes are obtained by averaging sample features. However, in MetaCluster, as the exact number of semantic prototypes is unknown, p_t is randomized and requires adjustments through forward propagation.

$$l_t(x_{t-1}, p_t) = \sigma \circ \begin{cases} \frac{p_t \cdot x_{t-1}^T}{\|x_{t-1}\| \cdot \|p_t\|} & (2a) \\ \text{LN}(p_t W_k^{(t)} \cdot (x_{t-1} W_q^{(t)})^T) + b_t & (2b) \\ e^{-\|p_t - x_{t-1}\| \otimes W_t / \tau + b_t} & (2c) \end{cases}$$

$l_t(x_{t-1}, p_t) : \mathbb{R}^{|x_{t-1}| \times v_{t-1}} \times \mathbb{R}^{|x_t| \times v_{t-1}} \rightarrow \mathbb{R}^{|x_t| \times |x_{t-1}|}$ represents the process of establishing “meta-aggregation,” which can be achieved through cosine similarity (eq. (2a)), attention mechanisms (eq. (2b)), LN is short for Layer Normalization), or L2 norm distance (eq. (2c)), among other techniques. Where τ represents the temperature coefficient. $W_q^{(t)} \in \mathbb{R}^{v_{n-1} \times d_k}$ and $W_k^{(t)} \in \mathbb{R}^{v_{n-1} \times d_k}$ are query and key weights for attentions. $W_t \in \mathbb{R}^{v_{t-1}}$ denote the weight matrices for the x_{n-1} ’s vectors respectively. $b_t \in \mathbb{R}$ is the bias. $\otimes$ is the element-wise multiply function. σ could be any kind of activation methods like relu, min-max-scale, tanh, softmax, or none.

The complete MetaCluster model is constructed through the stacking of MetaCluster layers, with slight modifications to the input of the first layer and the output of the final layer. The following formula is responsible for transforming any input X into an acceptable form $x_0 \in \mathbb{R}^{|x_0| \times v_0}$.

$$x_0 = \begin{cases} \text{Flatten}(\text{Conv2D}(X)), & X \in \mathbb{R}^{w \times h \times c} & (3a) \\ \text{Embedding}(X), & X \in \mathbb{N}^s & (3b) \\ \text{Reshape}_{\mathbb{R}^v \rightarrow \mathbb{R}^{1 \times v}}(X), & X \in \mathbb{R}^v & (3c) \end{cases}$$

There are generally three input modalities for cybersecurity classification problems: images, sequences, and feature sets. When the input $X \in \mathbb{R}^{w \times h \times c}$ represents images

(where w denotes width, h denotes height, and c denotes the number of channels), convolution and flattening operations will be employed for transformation. When the input $X \in \mathbb{N}^s$ represents sequences (with s being the sequence length), transformation is accomplished through word embedding or sentence embedding. When the input $X \in \mathbb{R}^v$ corresponds to feature sets (with v denoting the number of vectors), reshaping is utilized to convert $\mathbb{R}^v$ into $\mathbb{R}^{1 \times v}$.

The output of MetaCluster consists of a probability set $P_Y \in \mathbb{R}^{|Y|}$ for classification labels Y . Assuming this output is from the n -th layer of the MetaCluster, there are two methods to obtain P_Y (in Formula (4a) and Formula (4b)). Both of them can be activated using $\sigma(\cdot)$, with available activation options including identity, sigmoid, softmax, or others.

$$P_Y = \sigma \circ \begin{cases} l_n(x_{n-1}, p_Y), & \text{if } |x_{n-1}| = 1 & (4a) \\ f_n(l_n(x_{n-1}, p_Y) \cdot x_{n-1}), & \text{if } |x_{n-1}| > 1 & (4b) \end{cases}$$

The first method is the original PN approach, where, assuming the penultimate layer contains only one semantic prototype, $x_{n-1} \in \mathbb{R}^{1 \times v_n}$, the labels that surpass a certain threshold are determined solely by comparing the similarity between the input features x_{n-1} of the last layer and the prototypes $p_Y \in \mathbb{R}^{|Y| \times v_n}$ corresponding to the labels in Y .

The second method does not necessitate the assumption of having only one prototype but instead employs a function $f_n(\cdot) : \mathbb{R}^{|Y| \times v_{n-1}} \rightarrow \mathbb{R}^{|Y| \times 1}$ to transform the features into probability outputs. Here, $p_Y \in \mathbb{R}^{|Y| \times v_{n-1}}$ is still the label prototypes set.

B. Interpretability & Explainability

The answer to RQ2 implies the need to design metrics to measure the reliability and stability of the interpretation results. In fact, the interpretation process of MetaCluster can be decomposed into two stages: feature explanation and pattern interpretation. The feature explanation is used to get the

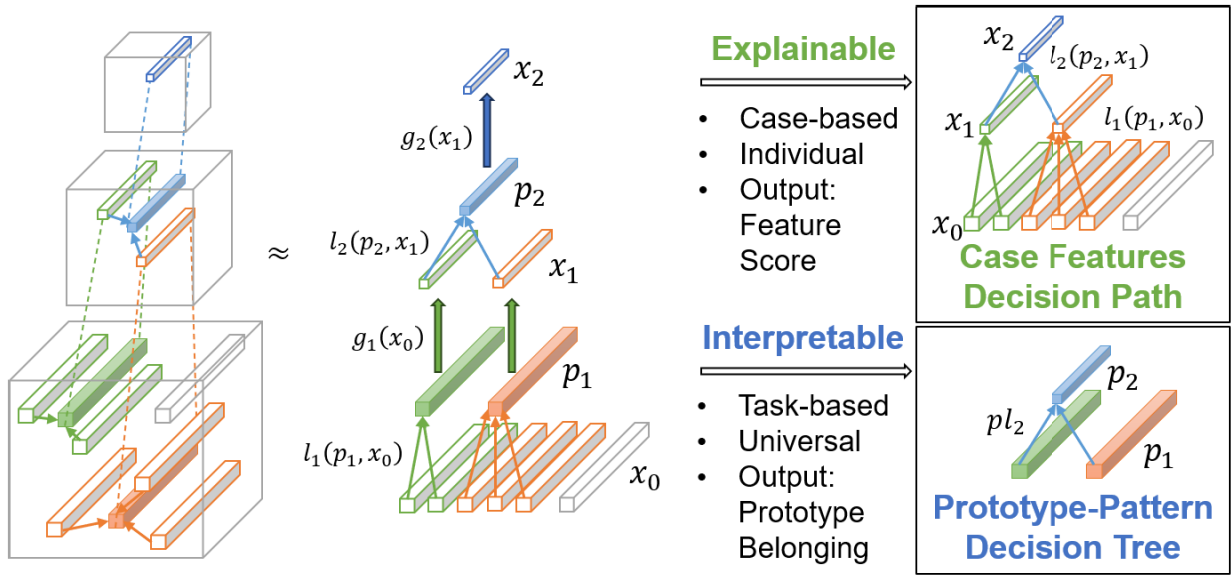


Fig. 2. The interpretation process of MetaCluster. Since semantic aggregation occurs in each MetaCluster layer, the structure of the model with softmax activation can be equated to a tree structure with level-by-level feature upward aggregation in the global view. The structure possesses both explainable and Interpretable properties. Explanation indicates that any feature can obtain a contribution to the classification result. Interpretation indicates that an invariant inference path can be obtained for any classification task.

interpretation results and the pattern interpretation is used to measure the reliability and stability of the interpretation results. The difference between this two explanations will be taken as an example of a simple two-layer model as shown in Fig. 2.

Feature explanation is the process of transforming all $l_t(x_{t-1}, p_t)$ by individually abstracting them and continuously linking them to form $x_0 \rightarrow x_n$. Although there is an embedding operation $f_t(\cdot)$ in the original MetaCluster model, the elements before and after the embedding can correspond to each other since each embedding only modifies the vector v_t and not the number of features $|x_{t-1}|$ or $|p_t|$. Then a one-way feature interpretation tree or graph with x_t as the node and $l_t(x_{t-1}, p_t)$ as the edge can be obtained. It is also possible to obtain an arbitrary label Y on the x_0 attention score $s(x_0) \in \mathbb{R}^{|x_0| \times |Y|}$, here the $P_Y \in \mathbb{R}^{|Y| \times 1}$ is the predictions of the model:

$$s(x_0) = l_1(x_0, p_1) \cdots l_t(x_{t-1}, p_t) \cdots l_n(x_{n-1}, p_Y) P_Y \quad (5)$$

Pattern interpretation achieves quantitative interpretability by constructing links to decision patterns. A simple approach to obtain highly reliable and stable interpretation measurements is to verify that the model can categorize feature prototypes for each class into the correct belongings. Also, considering that the explanation cannot be far-fetched, empty feature prototypes must be recognized as empty labels.

It is worth noting that for a given MetaCluster prediction model, where $p_t, f_t(\cdot), l_t(\cdot)$ are not varying with samples. In this way, if the p_t decision tree or graph can be constructed by constructing the p_t decision tree or graph for each layer of necklaces, it will be possible to represent the fundamentals and logic of the classification task. The formulation pl_t is the edge connecting the p_t samples and satisfies

the following equation:

$$pl_t = \begin{cases} g_1(x_Y), & \text{if } t = 1 \\ g_t(pl_{t-1}), & \text{otherwise} \end{cases} \quad (6a)$$

$$(6b)$$

where $x_Y \in \mathbb{R}^{|Y| \times w \times v}$ is the feature prototype of each label Y . Unlike label prototype p_Y in the last layer, x_Y emphasizes the feature prototypes obtained in the embedding space corresponding to the original x_0 without any embedding and aggregation. This concept is similar to the “average face” [53] in the field of face recognition, which can reflect the “average” characteristics of a classification. Because the number of x_Y and the number of labels Y are the same, as well as to build $|Y|$ decision paths to $|Y|$ categories, the model output $pl_n \in \mathbb{R}^{|Y| \times |Y|}$ of the x_Y prototypes is expected to be close to the unit matrix $I_{|Y| \times |Y|}$. At this time, each feature prototype has been precisely matched to each classification. Thus it exists MSE: eq. (7) so that the smaller $loss_{pl}$ is, the more accurate the x_Y and the model are.

$$loss_{pl} = \frac{\|pl_n - I_{|Y| \times |Y|}\|_2}{|Y|} \quad (7)$$

In particular, pl_t can also accept inputs of all-0 vectors or matrices, i.e., $[0]_{w \times v}$. In this case, the default is that the model accepts the task of predicting the null semantics and should output an all-0 prediction. Then the $loss_{pl}$ at this point will be adjusted to the following prediction process. The smaller $loss_{pl}^{(0)}$ is, the greater the model’s immunity to interference.

$$pl_t^{(0)} = \begin{cases} g_1([0]_{w \times v}), & \text{if } t = 1 \\ g_t(pl_{t-1}^{(0)}), & \text{otherwise} \end{cases} \quad (8a)$$

$$(8b)$$

$$loss_{pl}^{(0)} = \|pl_n^{(0)} - [0]_{|Y|}\|_2 = \|pl_n^{(0)}\|_2 \quad (9)$$

$loss_{pl}^{(0)}$ and $loss_{pl}$ can also serve as metrics for evaluating other classification models if they are not used in the training

TABLE I
DATASETS FOR DIFFERENT CLASSIFICATION TASKS

Task	Dataset	Class num	Train count	Test count
Malware	Drebin [56]	132	4961	552
ATT&CK	CTI-TTPs [20]	12+133	5858	651
Traffic	CIC-IDS-2017 [55]	14	55765	501881

process. When $loss_{pl}$ is smaller, the model is more able to differentiate the core semantic prototypes into distinct categories, proving a more reliable and believable prediction. A smaller $loss_{pl}^{(0)}$ signifies that the model is better able to resist external interference and noise.

IV. EXPERIMENTS

A. Experiment Settings

The experiments primarily focus on answering RQ1: Universal Classification Framework and RQ2: Quantitative Interpretation Assessment. RQ1 requires classification analysis that is multi-tasked and multi-input. RQ2 demands that the explanations and examinations in **Sect. III-B** exist for each experimental species. Thus, the experiments will be partitioned into several cybersecurity classification scenarios. They are malware classification with image input, ATT&CK classification with CTI text input, and malicious traffic classification with feature sets. The datasets for these three scenarios are given in **Table I**.

For the malware classification task, we used the Drebin datasets. Drebin classification involves identifying malicious programs from Android packages, primarily in binary format. To process the input data, we will utilize Markov image processing [54] in conjunction with eq. (3a).

ATT&CK classification dissects attack tactics and techniques defined by Mitre ATT&CK¹ from unstructured Cyber Threat Intelligence (CTI). To handle the unstructured CTIs, we'll employ a sequence-based approach and DistilBERT² embedding as outlined in eq. (3b).

For malicious traffic classification, the statistical feature dataset of CIC-IDS-2017 [55] was selected. This dataset is based on uniform features, and we'll employ eq. (3c) as the input processing method. In the data processing, we transform it to exclude the BENIGN type of traffic in favor of one-hot coding due to the excessive number and the lack of maliciousness of BENIGN tags. Specifically, when the prediction result is all zeros, it is considered consistent with the BENIGN label.

The experiments' general settings are summarized in **Table II**. The meta-cluster-layers' settings like "Meta num $|x_t|$ ", "Meta-cluster-layer num", and "Meta weight v_t " are chosen by the hyper-parameter optimization framework for TensorFlow models named keras_tuner.³ We will assess the classification results using metrics such as Precision, Recall, and F1, while also considering the model's size and processing speed. The interpretation of results will involve feature

TABLE II
GENERAL EXPERIMENT SETTINGS

Parameter	Value	Parameter	Value
Conv1D units	128	Conv1D kernel	3
Conv1D strides	1	Conv2D units	128
Conv2D kernel	(8,8)	Conv2D strides	(4,4)
Dense units	128	Attention units	128
LSTM units	128	eq. (2b) σ	Softmax
eq. (2a) σ	None/Relu	eq. (2c) σ	None
Best Meta num $ x_t $	128/12;128/5	Meta-cluster-layer num	1/1/1
Best Meta weight v_t	64/32/32	Learning rate	1e-4
Training epochs	≥ 40	GPU memory	24GB
CUDA version	11.3	Tensorflow version	2.5.0

attention maps or other visualization techniques by eq. (5) and eq. (8).

The DL and machine learning models in **Table III** will be used for experimental comparisons. The methods in the upper half of the table are traditional methods, while the methods in the lower half are XAIs. Due to the different types of inputs in the three experiments, not all methods will be used as comparisons for the same set of experiments.

B. Malware Classification

1) *Classification Analysis*: As Drebin is processed as Markov images for input, it primarily involves image classification techniques. The following methods have been selected in this sub-section, with each method utilizing the sigmoid($\cdot$) activation for outputting: ViT, MLP, MalPurifier, ResNet, Conv-BiLSTM, ProtoTree, ProtoPNet, *-PN, *-DNDT, and *-NeT.

The classification results of the aforementioned comparative methods and MetaCluster are presented in **Table IV**. In the case of MetaCluster, the implementation of $f_t(\cdot)$ is indicated within parentheses.

According to the results in the table, it can be observed that MetaCluster achieves a higher F1 score and less processing time. While most of the decision tree explanation models have reduced their accuracy compared to the original models. This advantage can be attributed to MetaCluster's meta-aggregation mechanism. On one hand, it helps compress model size to avoid redundant computation of similar features. On the other hand, it ensures that the features generated by $f_t(\cdot)$ can participate relatively "completely" in subsequent computations.

Simultaneously, it can be also observed that MetaCluster using Conv1D outperforms the model using Dense layers. Referring to the definitions of Query and Key in ViT, this difference may be attributed to MetaCluster's feature fusion based on the similarity of features, while Conv1D's fusion relies on spatial connectivity. In this classification task, a single pass of spatial correlation in image input may not independently provide all the necessary features for classification. Therefore, using Conv1D can better leverage the cooperative advantage of neighboring local features, leading to an improvement in classification performance.

2) *Feature Explanation*: **Fig. 3** provides explanations of Drebin's features when using MetaCluster (Dense), Conv-DNDT, ProtoPNet, ProtoTree, Conv-NeT and ViT-NeT.

¹<https://attack.mitre.org/>

²<https://huggingface.co/distilbert-base-uncased>

³https://tensorflow.google.cn/tutorials/keras/keras_tuner

TABLE III
THE EXPERIMENTAL COMPARISON METHODS

Model	Citation	is XAI?	Description
DT	[42]	No	Decision Tree utilizes a binary tree structure to build the classifier, where each node corresponds to a screening threshold for a feature.
RF	[48]	No	The optimal method identified by Verkerken et al. [48] in CICIDS 2017 for unknown attack detection
MalPurifier	[57]	No	MalPurifier uses a Denoising AutoEncoder (DAE)-based deep network to achieve recognition of obfuscated malware, which is considered a State-of-the-art (SOTA) approach at this time
LinearSVC	[58]	No	LinearSVC combines input features by weighting and summing them, and the resulting sum differentiates with the aid of offset values and hinge loss.
MLP	[59], [60]	No	Multi-Layer Perceptron employs stacked fully connected networks to perform the transformation of input features into the classification. [60] has used MLP networks as a way to identify adversarial malware.
CNN	[61]	No	We utilize TextCNN to aggregate adjacent text segments and apply a maximum pooling layer to produce a text vector.
BiLSTM	[62]	No	Utterance vectors are extracted by applying maximum pooling after contextualization with a BiLSTM.
ResNet	[63]	No	ResNet is a deep learning architecture that is characterized by the use of stacked convolutional neural networks. The key innovation in ResNet is residual connection, which is designed to address the vanishing gradient problem.
Transformer	[64]	No	The multi-layer Multi-Head self-Attention (MHA) networks for text classifications.
ViT	[65]	No	Vision Transformer is an innovative attention-based image classification method introduced by Google. It leverages attention mechanisms of the Transformer architecture and integrates them with the principles of residual neural networks.
TextRCNN	[66]	No	TextRCNN adopts a BiLSTM-CNN structure for classification. Following the BiLSTM output, a residual layer is introduced to enhance the signal.
Conv-BiLSTM	[67]	No	Conv-BiLSTM is a model that relies on convolutional input and distinguishes itself by incorporating a BiLSTM network.
BiLSTM-MHA	-	No	The BiLSTM initially contextualizes the text, and then the Multi-Head Attention (MHA) mechanism combines the word vectors to predict labels.
CtxAtt	[22]	No	Contextual Attention employs key semantics-based attention mechanisms and residual networks, distinguishing itself from MLP.
LIME	[16]	Yes	By morphing the samples and augmenting the data, LIME can explore whether arbitrary features can sway the model's output. Therefore, it has been used in the design of a variety of classification frameworks.
ProtoPNet	[51]	Yes	ProtoPNet simultaneously establishes semantic matches between multiple prototypes and convolutional blocks of inputs. It computes the highest similarity for each prototype weighted and summed to generate a probabilistic prediction of the category.
*-PN	[40]	Yes	Model with Prototype Network is an extension of the PN model. It accomplishes classification by transforming the last layer into a twin network, where class predictions are determined using the L2 distance and the softmax function.
*-DNDT	[31]	Yes	Model with Deep Neural Decision Tree is a classification method that extends the DNDT framework. It constructs a decision tree by matching and maximizing chunked prototypes extracted from the final layer.
ProtoPNet	[51]	Yes	Prototypical Part Network simultaneously establishes matches between multiple semantic prototypes and convolutional blocks of input data. The highest similarity score for each prototype is weighted and summed to create category prediction.
ProtoTree	[52]	Yes	Neural Prototype Trees construct a deep decision tree for convolutional-processed plots by matching feature prototypes layer by layer in a binary DT.
*-NeT	[22]	Yes	Model with Neural Tree Decoder (the original model is ViT-NeT) adds a prototype comparison to each layer of the original attention mechanism, and the maximum value of the comparison.
SATG	[20]	Yes	Self-Adversarial Topic Generation constructs a segmentation function to extract keywords and makes label predictions. The training process is adversarial and concludes when it yields the required classification bases.

TABLE IV

DREBIN CLASSIFICATION RESULT AFTER MARKOV IMAGE PROCESSING. THE SPEED IS MEASURED IN MS/STEP. SOME OF THE NAMES HAVE BEEN SHORTENED TO SAVE SPACE, THEY ARE: CRNN FOR CONV-BiLSTM AND MC FOR METACLUSTER

Model	Precision	Recall	F1	Size	Speed
ViT [65]	0.9715	0.8659	0.9157	74,756	30
MLP [60]	0.9681	0.9102	<u>0.9383</u>	2,101,637	8
ResNet [63]	0.9521	0.8641	0.9060	632,196	12
MalPurifier [57]	0.9314	0.7847	0.8516	140,678	136
CRNN [67]	0.9789	0.8424	0.9055	305,412	183
ViT-PN [40]	0.9176	0.8478	0.8813	91,396	30
ProtoPNet [51]	0.8963	0.6738	0.7691	45,697	76
ProtoTree [52]	0.7414	0.3904	0.5079	291,964	118
Conv-DNDT [31]	0.9252	0.7945	0.8547	66,962	9
Conv-NeT	0.9418	0.9109	0.9261	196,913	36
ViT-NeT [22]	0.9494	0.8731	0.9097	256,451	48
MC (Dense)	0.9841	0.8949	0.9374	80,864	7
MC (Conv1D)	0.9736	0.9335	0.9531	172,673	8
Disparity	$\uparrow 0.0055$	$\uparrow 0.0233$	$\uparrow 0.0148$	$\downarrow 91.78\%$	0

The figure uses the feature attention maps to show the output (eq. (5)) of the models for part of the classifications.

Each row in the subgraphs represents the feature focus of the model on a class of samples. Different rows correspond to the results of the seven malware classes as “Plankton, GinMaster, FakeDoc, FakeInstaller, Opfake, BaseBridge, and Kmin”. The more stable the interpretation method, the more similar the area of the feature interests in the same class, and the more significant the discrepancy between the patterns over different classes. Based on this hypothesis, it can be found that MetaCluster’s explanation possesses significant variability.

Although there is more than one feature attention under the same class, MetaCluster has more different distributions of features of interest for any class than Conv-DNDT, ProtoPNet, ProtoTree, and Conv-NeT. And the feature distributions between classes are highly consistent. Even based on prototype similarity measures, the feature maps of Conv-DNDT, ProtoPNet, and ProtoTree are not intuitive and easy to differentiate. This may be due to the fact that the prototypes of these three are only related to the convolved feature x_0 , which makes the attention relatively distracting and the visualization not obvious. On the contrary, ConvNeT and ViT-NeT’s attentions

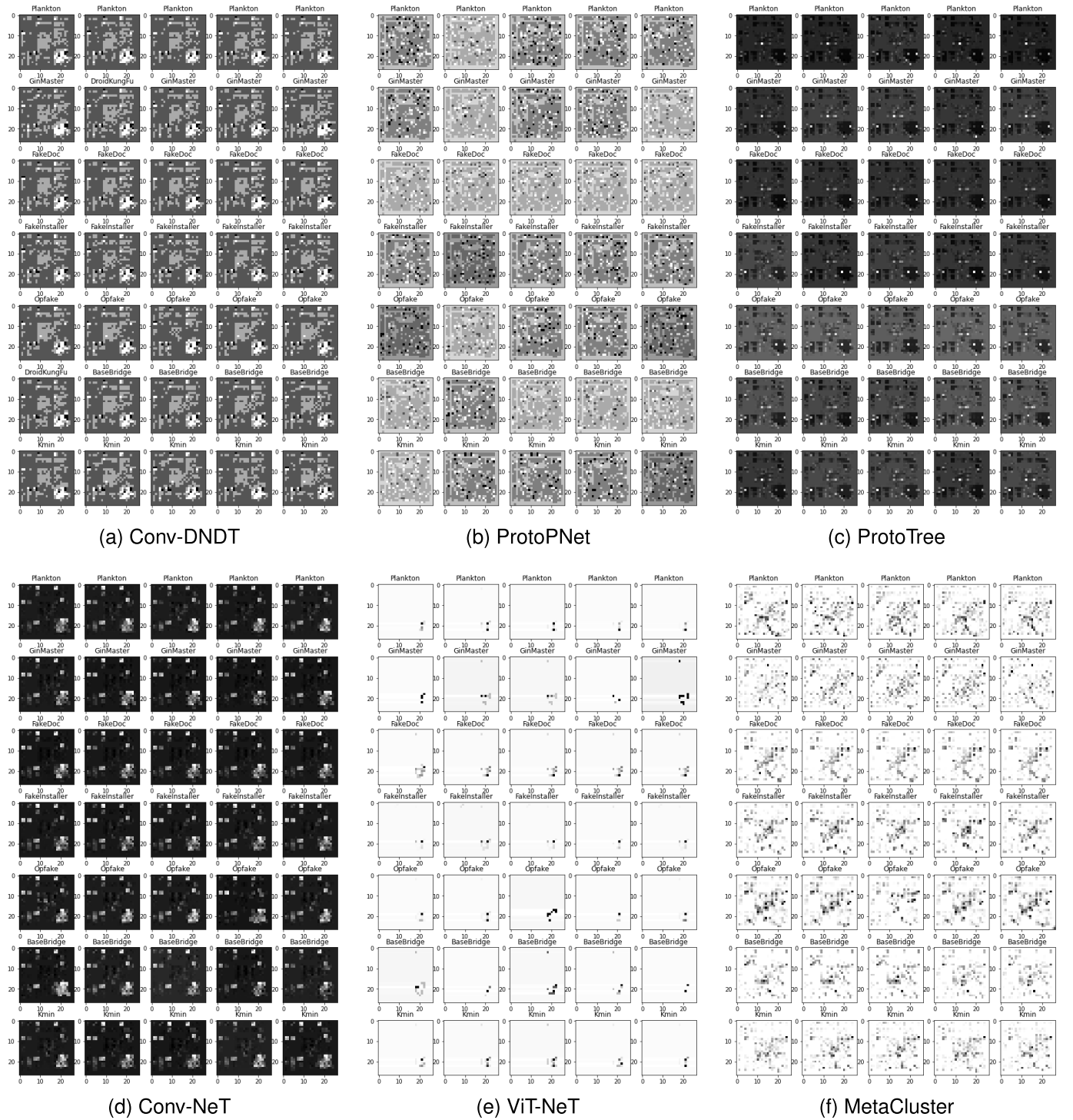


Fig. 3. The feature explanation for Drebin. Each row shows a type of different examples of image feature attention. A good feature attention is the one with large differences between classes and small differences within classes. This is reflected in the graph by the similarity of peers and significant differences in the same column. In this way, it seems that MetaCluster has relatively good feature attention.

are focused while the feature attention differentiation across classes is not as clear as MetaCluster.

This indicates that MetaCluster can better categorize features and generate a direct mapping of features to classification results, which has both between-class variation and within-class uniformity. Therefore, it can be considered that the decision process generated by MetaCluster is stable and

direct, which conforms to the repeatability principle of interpretability.

3) *Pattern Interpretation*: The x_Y used here is the result of averaging by class for samples in the training and testing sets: $x_Y = [\frac{x}{|x|} | x \in X, \text{model}(x) = y]_{y \in Y}$. For MetaCluster the output y is pl_n , and pl_n is replaced by $\text{model}(x_Y)$ for other models. eq. (7) also applies to other models as well. When

TABLE V

THE PATTERN INTERPRETATION'S $loss_{pl}$ FOR MALWARE CLASSIFICATIONS. IF $loss_{pl}$ OF A MODEL IS CLOSER TO 0, ITS PREDICTIONS OF THE CATEGORY PROTOTYPES FOR EACH CLASS ARE CLOSER TO THE UNIT MATRIX, INDICATING THAT IT IS THE MORE SUITABLE IN STABLE SEMANTIC GENERALIZATION

$loss_{pl}$ \ Dataset \ Model	Train	Test
ViT	0.4140 (± 0.3695)	3.1357 (± 3.5371)
MLP	0.1768 (± 0.2383)	2.1971 (± 2.5066)
ResNet	0.2917 (± 0.3613)	1.0985 (± 1.0166)
MalPurifier	0.6088 (± 0.3375)	2.3804 (± 2.3362)
Conv-BiLSTM	0.6350 (± 0.3563)	1.1439 (± 0.7296)
ViT-PN	0.3225 (± 0.4245)	0.6893 (± 0.4956)
ViT-NeT	0.4864 (± 0.3421)	2.0403 (± 2.1104)
ProtoTree	1.0329 (± 0.1976)	1.0376 (± 0.1897)
ProtoPNet	0.9527 (± 0.3223)	0.9240 (± 0.2976)
Conv-NeT	0.2508 (± 0.342)	1.6702 (± 1.8036)
Conv-DNDT	1.0377 (± 0.1862)	1.6367 (± 0.8658)
MetaCluster (Dense)	0.2147 (± 0.3152)	0.5768 (± 0.4737)
MetaCluster (Conv1D)	0.1798 (± 0.2957)	0.5634 (± 0.4854)
Disparity	$\uparrow 0.0030$ (± 0.0574)	$\downarrow 0.1259$ (± 0.0102)

TABLE VI

$loss_{pl}^{(0)}$ FOR DIFFERENT MODELS IN MALWARE CLASSIFICATION. A SMALLER $loss_{pl}^{(0)}$ SIGNIFIES THAT THE MODEL IS BETTER EQUIPPED TO RESIST EXTERNAL INTERFERENCE. SOME OF THE MODEL NAMES HAVE BEEN SHORTENED TO SAVE SPACE. THEY ARE CRNN FOR CONV-BiLSTM AND MC FOR METACLUSTER

Model	$loss_{pl}^{(0)}$	Model	$loss_{pl}^{(0)}$
ViT	7.7261 (± 0.3568)	MLP	5.4339 (± 0.1675)
ResNet	0.6168 (± 0.0434)	MalPurifier	5.3890 (± 0.0234)
CRNN	1.7560 (± 0.1427)	ViT-PN	0.3836 (± 0.0325)
ViT-NeT	4.7182 (± 0.2799)	ProtoTree	0.4411 (± 0.0376)
ProtoPNet	0.1690 (± 0.0126)	Conv-NeT	0.1743 (± 0.2899)
Conv-DNDT	2.6236 (± 0.0930)	MC (Dense)	0.1533 (± 0.0110)
MC (Conv1D)	0.2458 (± 0.0200)	Disparity	$\downarrow 0.0157$ (± 0.0016)

eq. (7) is small, the resulting matrix should look more like the unit matrix, so it is only necessary to compare this matrix to get an idea of how well the model fits x_Y .

Table V shows the $loss_{pl}$ for different models. The $loss_{pl}$ measures the ability of the model to correctly predict x_Y . The smaller $loss_{pl}$ is, the more the model is able to predict the corresponding categorization using the standard, desired semantics. In this example, MetaCluster obtains relatively small results, signaling MetaCluster's outstanding ability to grasp standard semantics in images. At the same time, MetaCluster's loss difference between the training and testing datasets is small, which can be assumed that MetaCluster has the ability to find core semantics and extensions.

Table VI is the $loss_{pl}^{(0)}$ prediction of each model for all classes in Drebin with all-zero matrix. The $loss_{pl}^{(0)}$ of ViT-PN, ProtoPNet, ProtoTree, and MetaCluster is below 0.5, while the other models are greater than 0.5. Meanwhile, the standard deviation of MetaCluster (Dense) is the smallest. It shows that MetaCluster is more resistant to noise interference.

C. ATT&CK Classification

1) *Classification Analysis*: The primary task in utilizing the ATT&CK textual dataset revolves around the generation of word vectors through the application of text embedding techniques. In this regard, DistilBERT⁴ is employed as the chosen method for text embedding. Given that sequences serve as input, conventional text classification methodologies are typically considered as baseline approaches for comparative purposes, including CNN, BiLSTM, TextRCNN, BiLSTM-MHA, Transformer, SATG, *-NeT, ProtoTree, ProtoPNet, *-DNDT.

Due to the different abstraction levels of tactics and techniques, MetaCluster will respectively participate in the experiment using Dense, Conv1D, and Gated Recurrent Unit (GRU) as its $f_t(\cdot)$. Since GRU causes all semantics to be displaced, only the aggregation-after-embedding approach (eq. (1a)) is considered for the implementation of MetaCluster when GRU is used as $f_t(\cdot)$.

The results of the experiments are shown in **Table VII**. It can be found that MetaCluster is able to achieve higher F1 values using fewer parameters and at a faster speed, both when it comes to tactics and techniques categorization. The possible reason for this is that the use of prototype aggregation can change the reasoning process to be simpler and more efficient. Among them, when classifying tactics, it can be found that TextRCNN-NeT and MetaCluster (GRU) using cyclic recursion and aggregation ideas are able to achieve higher F1, suggesting that models that aggregate full-text contextual semantics are able to express the abstraction of tactics more fully. On the contrary, MetaCluster with Conv1D and BiLSTM-MHA-NeT are able to obtain better results when categorizing technologies, suggesting that localized information is more able to dominate the recognition of technologies.

2) *Feature Explanation*: We enumerate the keyword extraction of a specific CTI text by various models. The keywords are identified by the rule that the normalized score is higher than 50%. The results are shown in **Table VIII**. The statements in the table are from the CTI report [68], and the TTPs tags to which it belongs are T1071: Application Layer Protocol and TA0011: Command and Control (C2). According to the semantic constraints of the tags provided by Mitre,⁵ the manually extracted keywords mainly contain DNS and C2.

From the results in the table, although most of the models provide sufficient attention to DNS and C2, some of the models that classify well, such as TextRCNN-Net, BiLSTM-MHA-Net, Conv-Net, etc., all produce redundant attention to other non-important words. The reason for this may be the use of CNN or RNN structures. Although complex models are able to synthesize the context, the semantics are shifted imperceptibly, thus making unimportant information tainted with important semantics as well.

Among the explanations of all models, SATG, Transformer-NeT, and MetaCluster do not focus on redundant information and have relatively good classification results, indicating that the methods based on the attention mechanism are able to

⁴<https://huggingface.co/distilbert-base-uncased>

⁵<https://attack.mitre.org/versions/v8/techniques/T1071/004/>

TABLE VII
ATT&CK CLASSIFICATION RESULTS FOR CYBER THREAT INTELLIGENCE. THE SPEED IS MEASURED IN MS/STEP

Model	Tactics			Techniques		
	Precision	Recall	F1	Size	Speed	Speed
CNN	0.8926	0.8003	0.8439	312,972	7	0.8155
BiLSTM [62]	0.8257	0.8078	0.8166	952,972	11	0.8398
TextRCNN [66]	0.8814	0.8168	0.8479	1,067,788	12	0.8728
Transformer [64]	0.8334	0.8271	0.8301	379,788	8	0.7676
BiLSTM-MHA	0.8385	0.7994	0.8184	1,232,780	12	0.8463
SATG [20]	0.8895	0.8258	0.8564	316,892	9	0.8879
Conv-DNDT [31]	0.6227	0.4578	0.5247	307,475	7	0.7665
ProtoPNet [51]	0.8884	0.6981	0.7813	313,022	8	0.9306
ProtoTree [52]	0.8744	0.7490	0.8068	953,077	13	0.5563
Conv-NeT	0.8899	0.8236	0.8554	544,364	20	0.8660
TextRCNN-NeT	0.8804	0.8454	0.8624	1,200,762	25	0.8917
Transformer-NeT [22]	0.8833	0.8260	0.8537	892,458	35	0.8235
BiLSTM-MHA-NeT	0.8771	0.8287	0.8521	1,745,450	39	0.8520
MetaCluster (Dense)	0.8381	0.8609	0.8493	53,100	8	0.7896
MetaCluster (Conv1D)	0.8588	0.8407	0.8496	92,988	8	0.7921
MetaCluster (GRU)	0.9105	0.8704	0.8899	137,580	9	0.8401
Disparity	$\uparrow 0.0301$	$\uparrow 0.0250$	$\uparrow 0.0275$	$\downarrow 88.54\%$	$\downarrow 64.00\%$	$\downarrow 0.0599$

TABLE VIII

THE NORMALIZED WORDS SCORES (EQ. (5)) FOR A SENTENCE IN [68]. THE LABELS FOR THE SENTENCE ARE T1071: APPLICATION LAYER PROTOCOL AND TA0011: COMMAND AND CONTROL (C2). THE **BOLDER SCORES OF WORDS** ARE HIGHER THAN 50%. MSE IS THE MEAN SQUARED ERROR BETWEEN THE HUMAN AND THE MODEL. IF THE GAP OR MSE BETWEEN THE HUMAN'S PREDICTION AND THE MODEL'S PREDICTION IS SMALL, IT MEANS THAT THE MODEL IS BETTER ABLE TO ACQUIRE KEYWORDS THAT ARE RECOGNIZED AS OFFENSIVE BY HUMANS

Scores \ Words													MSE
Models	[CLS]	helm	##int	##h	can	use	d	##ns	for	c2	.	[SEP]	
Human	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0	-
DNDT	0.0	0.33	0.0	0.33	0.0	0.67	0.67	1.0	1.0	1.0	1.0	1.0	1.94
SATG	0.06	0.01	0.06	0.01	0.03	0.0	0.88	1.0	0.06	0.15	0.04	0.05	0.87
ProtoPNet	0.18	0.29	0.18	0.05	0.45	0.0	0.69	1.0	0.20	0.75	0.73	0.13	1.05
ProtoTree	0.0	0.01	0.08	0.40	0.48	0.85	1.0	0.87	0.50	0.58	0.0	0.0	1.25
Conv-NeT	0.0	0.28	0.25	0.30	0.56	0.66	0.81	0.91	0.15	0.62	0.0	0.02	1.09
TextRCNN-NeT	0.79	0.43	0.54	0.96	0.55	0.0	0.10	0.75	0.63	1.0	0.48	0.48	2.01
Transformer-NeT	0.04	0.0	0.43	0.37	0.37	0.01	0.55	0.80	0.02	1.0	0.30	0.30	0.94
BiLSTM-MHA-NeT	0.0	0.25	0.68	0.66	0.66	0.59	0.62	0.67	0.75	1.0	0.71	0.72	1.89
MetaCluster (Dense)	0.18	0.0	0.0	0.0	0.0	0.02	0.09	0.94	0.0	1.0	0.0	0.0	0.93
MetaCluster (Conv1D)	0.0	0.0	0.0	0.0	0.0	0.30	0.35	1.0	0.0	0.87	0.0	0.0	0.73
MetaCluster (GRU)	0.0	0.0	0.0	0.0	0.0	0.08	0.58	1.0	0.06	0.87	0.0	0.0	0.45

form explanations efficiently and understandably. Throughout, MetaCluster's attention to all non-important words is almost 0.0, indicating that MetaCluster is able to form better explanations for text categorization.

More instances in text explanation of MetaCluster can be found in Fig. 4.

3) *Pattern Interpretation*: To obtain the average text x_Y for CTI, we counted the TF-IDF scores of the words (DistilBert's tokens) in all tactical categorizations and selected the top $|X|$ words in each category as the result. The tactical category was chosen because the number of samples in the tactical category is much larger than the technical, which would make the sampling results more natural.

Table IX shows the $loss_{pl}$ of the different models for the classification of tactics. The $loss_{pl}$ measures the ability of the model to correctly predict x_Y . The smaller $loss_{pl}$ is, the more the model is able to translate standard, expected CTI semantics into corresponding behavioral categories. In this example,

MetaCluster obtains relatively small results, indicating that MetaCluster has mastered the standard definition of ATT&CK.

Table X shows the $loss_{pl}^{(0)}$ results of predicting all-zero inputs to tactics using different models. From the results, we can see that *-NeT's model can partially suppress the abnormal prediction of the original model, for example, the average $loss_{pl}^{(0)}$ of BiLSTM-MHA is 0.7153, while BiLSTM-MHA-NeT's is 0.3669. However, the prediction of MetaCluster for all classifications is close to 0, reaching a $loss_{pl}^{(0)}$ of 0.0070. This indicates that MetaCluster can suppress the expression of classification without semantic input, making the model more reliable.

D. Traffic Classification

1) *Classification Analysis*: Due to the temporal statistical characteristics provided by the malicious traffic dataset, the following models with sigmoid activation will be chosen for

T1007: System Service Discovery

[CLS] hyper ##bro can list all services and their configurations . [SEP]

T1059: Command and Scripting Interpreter

[CLS] bad ##ne ##ws is capable of executing commands via cm ##d . ex ##e . [SEP]

T1082: System Information Discovery

[CLS] down ##paper collects the victim host name and serial number , and then sends the information to the c2 server . [SEP]

T1543: Create or Modify System Process

[CLS] hc ##dl ##oa ##der install ##s itself as a service for persistence . [SEP]

T1082: System Information Discovery

[CLS] pi ##sl ##oa ##der has a command to collect victim system information , including the system name and os version . [SEP]

T1106: Native API

[CLS] meta ##mo ##rf ##o has used native win ##ap ##i calls . [SEP]

T1105: Ingress Tool Transfer

[CLS] bond ##up ##date ##r can download or up ##load files from its c2 server . [SEP]

Fig. 4. Other examples for explanations of MetaCluster (Dense).

TABLE IX

THE PATTERN INTERPRETATION'S $loss_{pl}$ FOR ATT&CK TACTICS CLASSIFICATIONS. IF $loss_{pl}$ OF A MODEL IS CLOSER TO 0, ITS PREDICTIONS OF THE CATEGORY PROTOTYPES FOR EACH CLASS ARE CLOSER TO THE UNIT MATRIX, INDICATING THAT IT IS THE MORE SUITABLE IN STABLE SEMANTIC GENERALIZATION

$loss_{pl}$ \ Dataset	Train	Test
Model		
CNN	0.5025 (± 0.5000)	0.7007 (± 0.4691)
BiLSTM	0.3518 (± 0.3976)	0.5745 (± 0.4587)
TextRCNN	0.8041 (± 0.3904)	0.8121 (± 0.4201)
Transformer	0.7148 (± 0.2970)	0.6880 (± 0.3927)
BiLSTM-MHA	0.4918 (± 0.3552)	0.5482 (± 0.4147)
DNDT	1.0368 (± 0.3707)	1.0682 (± 0.3983)
SATG	0.6474 (± 0.4193)	0.7595 (± 0.3639)
ProtoPNet	0.5768 (± 0.3504)	0.7716 (± 0.2621)
ProtoTree	1.0512 (± 0.2383)	1.0427 (± 0.1647)
Conv-NeT	0.5204 (± 0.4343)	0.5877 (± 0.4599)
TextRCNN-NeT	0.5471 (± 0.4990)	0.7098 (± 0.4300)
Transformer-NeT	0.6306 (± 0.3478)	0.6670 (± 0.3898)
BiLSTM-MHA-NeT	0.5949 (± 0.4458)	0.5614 (± 0.4179)
MetaCluster (Dense)	0.4183 (± 0.2243)	0.5788 (± 0.3315)
MetaCluster (Conv1D)	0.4631 (± 0.4424)	0.5297 (± 0.4319)
MetaCluster (GRU)	0.2502 (± 0.3801)	0.4425 (± 0.4512)
Disparity	$\downarrow 0.1016$ (± 0.0175)	$\downarrow 0.1057$ (± 0.0365)

comparison: DT, RF, LinerSVC, MLP, CtxAtt, DNDT and *-NeT.

Table XI shows the prediction metrics of different models. From the results in the table, it can be seen that DT and RF are the most powerful models for this dataset, followed by MetaCluster. The reason for this may be that DT is more suitable for distinguishing the decision boundaries of the features. MetaCluster, although not as good as DT, still

TABLE X

$loss_{pl}^{(0)}$ FOR DIFFERENT MODELS IN ATT&CK CLASSIFICATION. A SMALLER $loss_{pl}^{(0)}$ SIGNIFIES THAT THE MODEL IS BETTER EQUIPPED TO RESIST EXTERNAL INTERFERENCE. SOME OF THE MODEL NAMES HAVE BEEN SHORTENED TO SAVE SPACE, THEY ARE RCNN FOR TEXTRCNN, RANN FOR BiLSTM-MHA, AND MC FOR METACLUSTER

Model	$loss_{pl}^{(0)}$	Model	$loss_{pl}^{(0)}$
DNDT	1.1768 (± 0.2537)	SATG	0.9071 (± 0.1503)
ProtoPNet	0.0942 (± 0.0236)	ProtoTree	0.9809 (± 0.2432)
Conv-NeT	1.0383 (± 0.2479)	RCNN-NeT	1.5601 (± 0.3525)
Trans-NeT	0.8564 (± 0.2294)	RANN-NeT	0.3669 (± 0.0975)
CNN	1.6122 (± 0.0166)	BiLSTM	1.3428 (± 0.0650)
RCNN	1.6910 (± 0.0564)	Transformer	2.0573 (± 0.3691)
RANN	0.7153 (± 0.0894)	MC (Dense)	0.0298 (± 0.0077)
MC (Conv1d)	0.1741 (± 0.0477)	MC (GRU)	0.0070 (± 0.0017)
Disparity	$\downarrow 0.0872$ (± 0.0219)		

TABLE XI

TRAFFIC CLASSIFICATION RESULTS. THE SPEED IS MEASURED BY MS/STEP. MC IS SHORT FOR METACLUSTER

Model	Precision	Recall	F1	Size	Speed
DT [42]	0.9975	0.9975	0.9975	311	1
RF [48]	0.9982	0.9977	0.9980	32,042	5
LinearSVC [58]	0.7611	0.7611	0.7611	1,106	1
MLP [59]	0.9350	0.9349	0.9350	11,918	4
CtxAtt [22]	0.9588	0.9648	0.9618	21,615	5
DNDT [31]	0.8557	0.7413	0.7944	133,010	5
MLP-NeT [22]	0.9825	0.9771	0.9798	119,871	9
MC (Dense)	0.9886	0.9887	0.9887	3,664	5
Disparity	$\downarrow 0.0096$	$\downarrow 0.0090$	$\downarrow 0.0093$	$\downarrow 88.57\%$	0

maintains higher F1, lower space and time consumption among other deep learning methods.

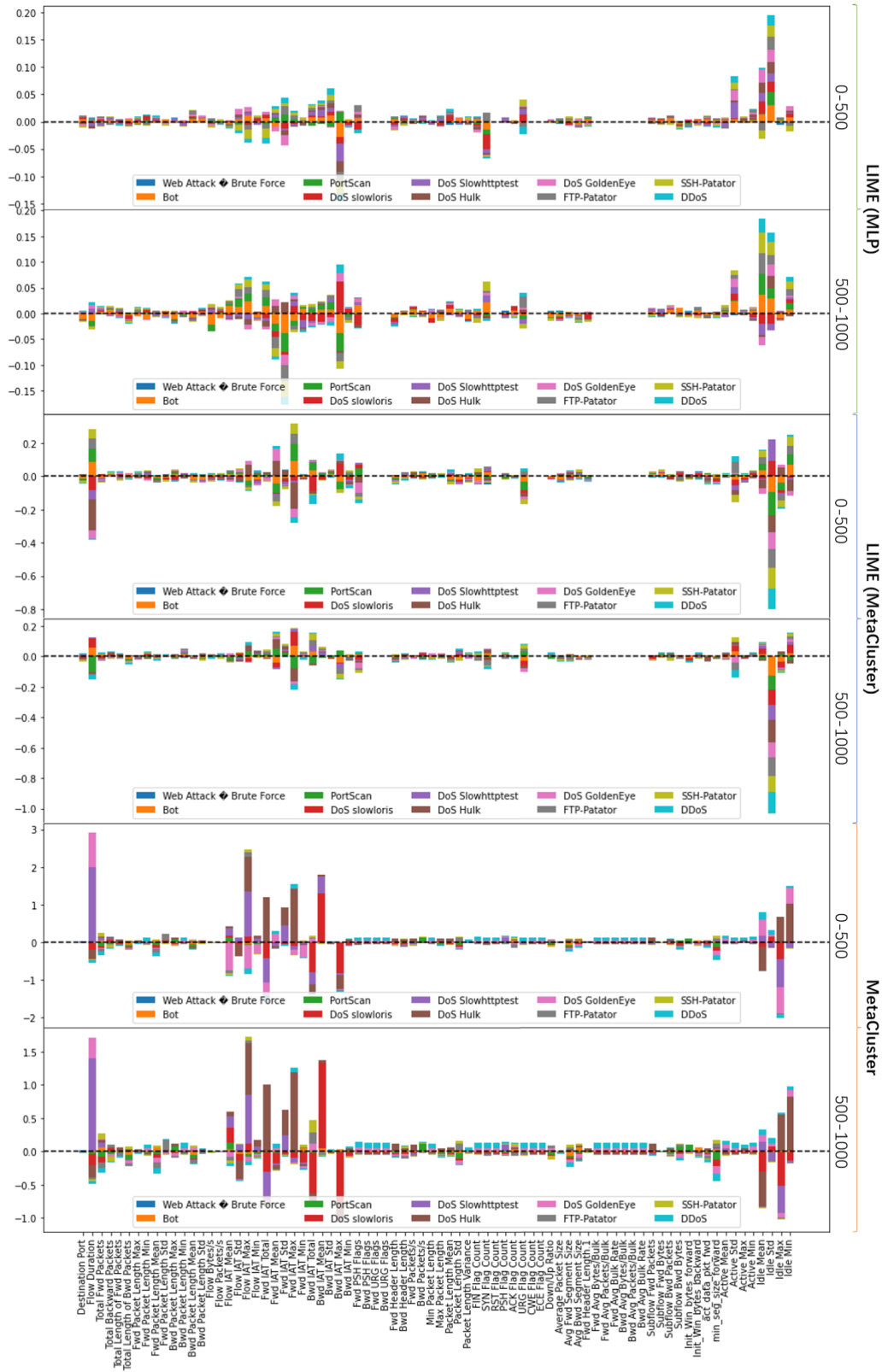


Fig. 5. The importance of categorical features in traffic dataset. The different colored bars are how much attention the model pays to different features of the same classification. If the score is much greater than 0, it means that the feature is relevant and more important to the classification.

2) *Feature Explanation:* Since MetaCluster will reshape $X \in \mathbb{R}^v$ to become $x_0 \in \mathbb{R}^{1 \times v}$, and eq. (5) will place all of the attention on the first dimension of x_0 , this inevitably

results in the use of the original x_0 with only one score left. This will lead to not being able to get the attention of the features in v , and therefore the x_0 transformation is needed.

One approach, like in LIME [16], is to break up the features into different sets and use these sets to see if the predictions are correct. Therefore, we use the Mask unit matrix $I_{v \times v}$ and the original x_0 to multiply by element, and the individual features of x_0 are broken up into different sets $x_f \in \mathbb{R}^{v \times v}$ to achieve the selection of important features. The formula is as follows:

$$x_f = x_0 \otimes I_{v \times v} = \begin{bmatrix} x_0^{(1)} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & x_0^{(v)} \end{bmatrix} \quad (10)$$

At this point, we then substitute both x_0 and x_f into MetaCluster to obtain the desired classification $y_0 \in \mathbb{R}^{1 \times l}$ and the classification results of independent features $y_f \in \mathbb{R}^{v \times l}$, as shown in eq. (11). We want that the y_f of the high importance x_f should be as close as possible to y_0 , so the score $s_f \in \mathbb{R}^{v \times 1}$ for each feature is calculated using eq. (12), where $\frac{\sum_v(y_f)}{v} \in \mathbb{R}^{1 \times l}$ is the base score for each class (cf. the y_{base} definition in [17]). $\text{abs}(\cdot)$ takes absolute values for the normalized x_0 . Under this requirement, when s_f is positive and has a large absolute value, it means that the feature is effective for y_0 classification. When s_f is close to 0, it means that the feature has no effect. When s_f is negative and the absolute value is large, the feature may helpful for non- y_0 classification.

$$y_0 = \text{MetaCluster}(x_0), y_f = \text{MetaCluster}(x_f) \quad (11)$$

$$s_f = \text{abs}(\text{LN}(x_f)) \text{ReLU}(y_f - \frac{\sum_v(y_f)}{v})(y_0 - \frac{\sum_l(y_0)}{l})^T \quad (12)$$

Fig. 5 shows how much attention LIME and MetaCluster pay to each feature. The left and right columns respectively show the average feature importance for each category for the 0th to 500th and the 500th to 1000th test samples. The top, middle and bottom rows are the feature importance scores obtained using the LIME [16] on MLP, LIME on MetaCluster and MetaCluster itself. It can be found that compared to LIME, the method in this paper is able to obtain a more obvious and stable level of feature attention, as well as a more obvious differentiation of features in different classes. It shows that the feature explanation of MetaCluster is stable and obvious.

3) *Pattern Interpretation*: Since a statistical feature dataset was chosen, it is assumed that the features of each classification obey a polynomial normal distribution. The expectation of the distribution, i.e., the average of the features of each sample, is considered as the “average feature prototype” x_Y for each classification. Bringing this feature prototype into the model prediction yields the degree of model fitness for each class of features. The closer the prediction is to a unit matrix, the more robust and reliable the model is.

Table XII shows the loss_{pl} profile of the traffic. The smaller the loss_{pl} , the more the model is able to correspond the average features from the normal distribution to the malicious categorized species. In this case, MetaCluster unsurprisingly obtains relatively small results, demonstrating the ability to understand standard semantics.

From Table XIII, it can be seen that MetaCluster’s $\text{loss}_{pl}^{(0)}$ is the closest to 0 when predicting null semantics, indicating that MetaCluster can resist the noise effect well.

TABLE XII

THE PATTERN INTERPRETATION’S loss_{pl} FOR TRAFFIC CLASSIFICATIONS. IF loss_{pl} OF A MODEL IS CLOSER TO 0, ITS PREDICTIONS OF THE CATEGORY PROTOTYPES FOR EACH CLASS ARE CLOSER TO THE UNIT MATRIX, INDICATING THAT IT IS THE MORE SUITABLE IN STABLE SEMANTIC GENERALIZATION

loss_{pl} \ Dataset \ Model	Train	Test
DT	1.3132 (± 0.3642)	1.2122 (± 0.4949)
RF	0.9932 (± 0.1015)	0.9197 (± 0.1956)
LinearSVC	0.6061 (± 0.6999)	0.5051 (± 0.6776)
MLP	0.5051 (± 0.6776)	0.5051 (± 0.6776)
CtxAtt	0.6306 (± 0.6779)	0.5400 (± 0.6501)
DNDT	1.2022 (± 0.2335)	1.1846 (± 0.2299)
MLP-NeT	0.4197 (± 0.4956)	0.4515 (± 0.5409)
MetaCluster	0.3569 (± 0.4241)	0.3581 (± 0.4386)
Disparity	$\downarrow 0.0628$ (± 0.0715)	$\downarrow 0.0934$ (± 0.1023)

TABLE XIII

$\text{loss}_{pl}^{(0)}$ FOR DIFFERENT MODELS IN TRAFFIC CLASSIFICATION. A SMALLER $\text{loss}_{pl}^{(0)}$ SIGNIFIES THAT THE MODEL IS BETTER EQUIPPED TO RESIST EXTERNAL INTERFERENCE

Model	$\text{loss}_{pl}^{(0)}$	Model	$\text{loss}_{pl}^{(0)}$
DT	1.0000 (± 0.2575)	RF	0.4552 (± 0.0985)
LinerSVC	1.0000 (± 0.2575)	MLP	1.5972 (± 0.3540)
CtxAtt	1.7321 (± 0.4103)	DNDT	0.6064 (± 0.1423)
MLP-NeT	0.9986 (± 0.2560)	MetaCluster	0.4451 (± 0.1066)
Disparity	$\downarrow 0.0101$ (± 0.0081)		

V. CONCLUSION

Addressing the need for a universal, stable, convenient, and efficient interpretable framework in cybersecurity, this paper introduces MetaCluster, a generic classification interpreter framework based on prototype semantic aggregation. MetaCluster filters, combines, and expresses classification semantics by generating feature prototypes at different granularity levels and constructing dynamic graph learning layers. We validate the applicability of MetaCluster to multiple interpretable classification tasks through three sets of experiments: malware, ATT&CK, and malicious traffic classification.

According to the experimental results, MetaCluster outperforms other deep learning models by achieving more accurate, stable, and explanatory classification predictions with lower time and space consumption. Specifically, in the classifications of malware, attack tactics, attack techniques, and malicious traffic, MetaCluster improves F1 values by 0.0148, 0.0275, 0.0023, and -0.0093 , reduces parameter count consumption by 91.78%, 88.54%, 79.52%, and 88.57%, and increases operation speed by 0.0%, 64.00%, 71.37%, and 0.0%, respectively. Additionally, MetaCluster can provide the importance of image, text, and statistical features, and it effectively resists noise interference on classification. On the malware, the tactics, and the traffic dataset, the Mean Squared Error (MSE) of prediction and expectation for the standard category prototypes is reduced by an average of 0.0099, 0.1020, and 0.0903, respectively. And the MSE on non-semantic predictions is reduced by 0.0157, 0.0872, and 0.0101.

In summary, MetaCluster is a self-interpreting lightweight classification model designed to offer interpretation and stability testing for various cybersecurity tasks. It is well-suited for deployment in low-reliability cyberattack scenarios.

REFERENCES

- [1] Fortinet. (Nov. 2023). *Cyber Threat Predictions for 2023*. [Online]. Available: <https://www.fortinet.com/content/dam/fortinet/assets/white-papers/wp-threat-prediction-2023.pdf>
- [2] J. Zhao, M. Shao, H. Wang, X. Yu, B. Li, and X. Liu, "Cyber threat prediction using dynamic heterogeneous graph learning," *Knowledge-Based Syst.*, vol. 240, Mar. 2022, Art. no. 108086.
- [3] N. Sun et al., "Cyber threat intelligence mining for proactive cybersecurity defense: A survey and new perspectives," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 3, pp. 1748–1774, 3rd Quart., 2023.
- [4] B. Tang et al., "Advanced persistent threat intelligent profiling technique: A survey," *Comput. Electr. Eng.*, vol. 103, Oct. 2022, Art. no. 108261.
- [5] D. Li, Q. Li, Y. Ye, and S. Xu, "Arms race in adversarial malware detection: A survey," *ACM Comput. Surveys*, vol. 55, no. 1, pp. 1–35, Nov. 2021.
- [6] Y. Jakhotiya, H. Patil, J. Rawlani, and D. S. B. Mane, "Adversarial attacks on transformers-based malware detectors," 2022, *arXiv:2210.00008*.
- [7] Y. Du, B. Prebot, X. Xi, and C. Gonzalez, "A cyber-war between bots: Human-like attackers are more challenging for defenders than deterministic attackers," in *Proc. Annu. Hawaii Int. Conf. Syst. Sci.*, 2023, pp. 856–865.
- [8] M. Beechey, K. G. Kyriakopoulos, and S. Lambrotharan, "Evidential classification and feature selection for cyber-threat hunting," *Knowledge-Based Syst.*, vol. 226, Aug. 2021, Art. no. 107120.
- [9] M. M. Alani and A. I. Awad, "PAIRED: An explainable lightweight Android malware detection system," *IEEE Access*, vol. 10, pp. 73214–73228, 2022.
- [10] M. Kinkead, S. Millar, N. McLaughlin, and P. O’Kane, "Towards explainable CNNs for Android malware detection," *Proc. Comput. Sci.*, vol. 184, pp. 959–965, 2021.
- [11] J. C. S. Reis, A. Correia, F. Murai, A. Veloso, and F. Benevenuto, "Explainable machine learning for fake news detection," in *Proc. 10th ACM Conf. Web Sci.* New York, NY, USA: Association for Computing Machinery, 2019, pp. 17–26.
- [12] H. Suryotrisongko, Y. Musashi, A. Tsuneda, and K. Sugitani, "Robust botnet DGA detection: Blending XAI and OSINT for cyber threat intelligence sharing," *IEEE Access*, vol. 10, pp. 34613–34624, 2022.
- [13] I. Psychoula, A. Gutmann, P. Mainali, S. H. Lee, P. Dunphy, and F. Petitcolas, "Explainable machine learning for fraud detection," *Computer*, vol. 54, no. 10, pp. 49–59, Oct. 2021.
- [14] P. R. G. Hernandez et al., "Phishing detection using URL-based XAI techniques," in *Proc. IEEE Symp. Series Comput. Intell. (SSCI)*, 2021, pp. 1–6.
- [15] Z. A. E. Houda, B. Brik, and L. Khoukhi, "Why should i trust your IDS?: An explainable deep learning framework for intrusion detection systems in Internet of Things networks," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 1164–1176, 2022.
- [16] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*. New York, NY, USA: Association for Computing Machinery, 2016, pp. 1135–1144.
- [17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.* Red Hook, NY, USA: Curran Associates, 2017, pp. 4768–4777.
- [18] B. Mittelstadt, C. Russell, and S. Wachter, "Explaining explanations in AI," in *Proc. Conf. Fairness, Accountability, Transparency*. New York, NY, USA: Association for Computing Machinery, Jan. 2019, pp. 279–288.
- [19] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Mach. Intell.*, vol. 1, no. 5, pp. 206–215, May 2019.
- [20] W. Ge, J. Wang, T. Lin, B. Tang, and X. Li, "Explainable cyber threat behavior identification based on self-adversarial topic generation," *Comput. Secur.*, vol. 132, Sep. 2023, Art. no. 103369.
- [21] S. Islam et al., "A comprehensive survey on applications of transformers for deep learning tasks," *Exp. Syst. Appl.*, vol. 241, May 2024, Art. no. 122666.
- [22] S. Kim, J. Nam, and B. C. Ko, "ViT-NeT: Interpretable vision transformers with neural tree decoder," in *Proc. 39th Int. Conf. Mach. Learn.*, vol. 162, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds. Jul. 2022, pp. 11162–11172.
- [23] F. Ullah, A. Alsirhani, M. M. Alshahrani, A. Alomari, H. Naeem, and S. A. Shah, "Explainable malware detection system using transformers-based transfer learning and multi-model visual representation," *Sensors*, vol. 22, no. 18, p. 6766, Sep. 2022.
- [24] M. Q. Li, B. C. M. Fung, P. Charland, and S. H. H. Ding, "I-MAD: Interpretable malware detector using galaxy transformer," *Comput. Secur.*, vol. 108, Sep. 2021, Art. no. 102371.
- [25] W. Ge and J. Wang, "SeqMask: Behavior extraction over cyber threat intelligence via multi-instance learning," *Comput. J.*, vol. 67, no. 1, pp. 253–273, Jan. 2024.
- [26] C. Liu, J. Wang, and X. Chen, "Threat intelligence ATT&CK extraction based on the attention transformer hierarchical recurrent neural network," *Appl. Soft Comput.*, vol. 122, Jun. 2022, Art. no. 108826.
- [27] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds. Red Hook, NY, USA: Curran Associates, 2020, pp. 1877–1901.
- [28] Z. Du et al., "GLM: General language model pretraining with autoregressive blank infilling," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 320–335.
- [29] B. Mahbooba, M. Timilsina, R. Sahal, and M. Serrano, "Explainable artificial intelligence (XAI) to enhance trust management in intrusion detection systems using decision tree model," *Complexity*, vol. 2021, pp. 1–11, Jan. 2021.
- [30] S. Wang et al., "TrafficAV: An effective and explainable detection of mobile malware behavior using network traffic," in *Proc. IEEE/ACM 24th Int. Symp. Quality Service (IWQoS)*, Jun. 2016, pp. 1–6.
- [31] Y. Yang, I. G. Morillo, and T. M. Hospedales, "Deep neural decision trees," 2018, *arXiv:1806.06988*.
- [32] H. Asad and I. Gashi, "Dynamical analysis of diversity in rule-based open source network intrusion detection systems," *Empirical Softw. Eng.*, vol. 27, no. 1, p. 4, Jan. 2022.
- [33] S. Guruprasad and R. D’Souza, "Development of an evolutionary framework for autonomous rule creation for intrusion detection," in *Proc. IEEE 6th Int. Conf. Adv. Comput. (IACC)*, Feb. 2016, pp. 534–538.
- [34] N. Fallahi, A. Sami, and M. Tajbakhsh, "Automated flow-based rule generation for network intrusion detection systems," in *Proc. 24th Iranian Conf. Electr. Eng. (ICEE)*, May 2016, pp. 1948–1953.
- [35] R. Muthuregumathan and S. R. Rajesh, "Efficient snort rule generation using evolutionary computing for network intrusion detection," in *Proc. 1st Int. Conf. Comput. Intell., Commun. Syst. Netw.*, Jul. 2009, pp. 336–341.
- [36] F. Charmet et al., "Explainable artificial intelligence for cybersecurity: A literature survey," *Ann. Telecommun.*, vol. 77, pp. 789–812, Oct. 2022.
- [37] M. Zolnari, Z. Yang, K. Khan, R. Jain, and N. Meskin, "TRUST XAI: Model-agnostic explanations for AI with a case study on IIoT security," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 2967–2978, Feb. 2023.
- [38] E. Tjoa and C. Guan, "A survey on explainable artificial intelligence (XAI): Toward medical XAI," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 11, pp. 4793–4813, Nov. 2021.
- [39] M. Al-Shouha and G. Szucs, "PIC-XAI: Post-hoc image captioning explanation using segmentation," in *Proc. IEEE 17th Int. Symp. Appl. Comput. Intell. Informat. (SACI)*, May 2023, pp. 000033–000038.
- [40] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Advances in Neural Information Processing Systems*, vol. 30, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Red Hook, NY, USA: Curran Associates, 2017.
- [41] Y. Zhang, P. Tino, A. Leonardis, and K. Tang, "A survey on neural network interpretability," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 5, no. 5, pp. 726–742, Oct. 2021.
- [42] A. J. Myles, R. N. Feudale, Y. Liu, N. A. Woody, and S. D. Brown, "An introduction to decision tree modeling," *J. Chemometrics*, vol. 18, no. 6, pp. 275–285, Jun. 2004.
- [43] K. P. Murphy, "Naive Bayes classifiers," *Univ. Brit. Columbia*, vol. 18, no. 60, pp. 1–8, 2006.
- [44] M. L. Puterman, "Chapter 8 Markov decision processes," in *Stochastic Models (Handbooks in Operations Research and Management Science)*, vol. 2. Amsterdam, The Netherlands: Elsevier, 1990, pp. 331–434.

- [45] F. James, "Monte Carlo theory and practice," *Rep. Prog. Phys.*, vol. 43, no. 9, p. 1145, Sep. 1980.
- [46] P. Barnard, N. Marchetti, and L. A. DaSilva, "Robust network intrusion detection through explainable artificial intelligence (XAI)," *IEEE Netw. Lett.*, vol. 4, no. 3, pp. 167–171, Sep. 2022.
- [47] A. Kuppa and N.-A. Le-Khac, "Adversarial XAI methods in cybersecurity," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4924–4938, 2021.
- [48] M. Verkerken et al., "A novel multi-stage approach for hierarchical intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 3915–3929, Sep. 2023.
- [49] R. Balestrieri, "Neural decision trees," 2017, *arXiv:1702.07360*.
- [50] S. R. Buló and P. Kotschieder, "Neural decision forests for semantic image labelling," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 81–88.
- [51] C. Chen, O. Li, A. Barnett, J. Su, and C. Rudin, "This looks like that: deep learning for interpretable image recognition," 2018, *arXiv:1806.10574*.
- [52] M. Nauta, R. van Bree, and C. Seifert, "Neural prototype trees for interpretable fine-grained image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 14933–14943.
- [53] T. Valentine, "A unified account of the effects of distinctiveness, inversion, and race in face recognition," *Quart. J. Experim. Psychol. Sect. A*, vol. 43, no. 2, pp. 161–204, May 1991.
- [54] B. Yuan, J. Wang, P. Wu, and X. Qing, "IoT malware classification based on lightweight convolutional neural networks," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3770–3783, Mar. 2022.
- [55] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.
- [56] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck, "DREBIN: Effective and explainable detection of Android malware in your pocket," in *Proc. NDSS*, Feb. 2014, pp. 1–15.
- [57] Y. Zhou, G. Cheng, Z. Chen, and S. Yu, "MalPurifier: Enhancing Android malware detection with adversarial purification against evasion attacks," 2023, *arXiv:2312.06423*.
- [58] M. Hearst, S. Dumais, E. Osuna, J. Platt, and B. Scholkopf, "Support vector machines," *IEEE Intell. Syst. Appl.*, vol. 13, no. 4, pp. 18–28, Aug. 1998.
- [59] H. Taud and J. Mas, *Multilayer Perceptron (MLP)*. Cham, Switzerland: Springer, 2018, pp. 451–455.
- [60] K. Grosse, N. Papernot, P. Manoharan, M. Backes, and P. McDaniel, "Adversarial examples for malware detection," in *Proc. Comput. Secur. ESORICS*, S. N. Foley, D. Gollmann, and E. Sneekenes, Eds. Cham, Switzerland: Springer, 2017, pp. 62–79.
- [61] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, A. Moschitti, B. Pang, and W. Daelemans, Eds. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1746–1751. [Online]. Available: <https://aclanthology.org/D14-1181>
- [62] Y. You et al., "TIM: Threat context-enhanced TTP intelligence mining on unstructured threat data," *Cybersecurity*, vol. 5, no. 1, p. 3, Dec. 2022.
- [63] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [64] N. Rani, B. Saha, V. Maurya, and S. K. Shukla, "TTPHunter: Automated extraction of actionable intelligence as TTPs from narrative threat reports," in *Proc. Australas. Comput. Sci. Week*. New York, NY, USA: Association for Computing Machinery, Jan. 2023, pp. 126–134.
- [65] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [66] X. Sun, L. Li, L. Bo, X. Wu, Y. Wei, and B. Li, "Automatic software vulnerability classification by extracting vulnerability triggers," *J. Software, Evol. Process*, vol. 36, no. 2, Feb. 2024, Art. no. e2508.
- [67] H. Alkahtani and T. H. H. Aldhyani, "Botnet attack detection by using CNN-LSTM model for Internet of Things applications," *Secur. Commun. Netw.*, vol. 2021, pp. 1–23, Sep. 2021.
- [68] R. Falcone and B. Lee. (Mar. 2017). *The Oilrig Campaign: Attacks on Saudi Arabian Organizations Deliver Helminth Backdoor*. [Online]. Available: <https://unit42.paloaltonetworks.com/the-oilrig-campaign-attacks-on-saudi-arabian-organizations-deliver-helminth-backdoor/>



Wenhan Ge received the B.S. and M.S. degrees in computer science and technology from Sichuan University, China, where he is currently pursuing the Ph.D. degree. His research interests include cyber threat intelligence, natural language processing, and explainable artificial intelligence.



Zeyuan Cui is currently pursuing the M.S. degree with Sichuan University, Chengdu, China. His research interests include cyber threat analysis, machine learning in security contexts, and intelligent decision-making.



Junfeng Wang received the M.S. degree in computer application technology from Chongqing University of Posts and Telecommunications, Chongqing, in 2001, and the Ph.D. degree in computer science from the University of Electronic Science and Technology of China, Chengdu, in 2004. From July 2004 to August 2006, he was a Post-Doctoral Researcher with the Institute of Software, Chinese Academy of Sciences. Since August 2006, he has been a Professor with the College of Computer Science, Sichuan University.

His recent research interests include network and information security, spatial information networks, and data mining. He serves as an associate editor for several international journals.



Binhui Tang is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Sichuan University. She is an Associate Professor with Chengdu Jincheng College. Her recent research interests include APT analysis, cyber threat intelligence mining, cyber-space attacks, and defensive confrontation.



Xiaohui Li (Member, IEEE) received the B.S. and Ph.D. degrees in computer science from the College of Computer Science, Sichuan University, Chengdu, China, in 2012 and 2017, respectively. She is currently a Teacher with the School of Cyber Science and Engineering, Sichuan University. Her research interests include network and information security, such as wireless networks, satellite networks, the Internet of Things, and cyber threat intelligence, with an emphasis on the design of efficient transport protocols, routing protocols, and intelligence analysis.